ljopt: Translation Validation and IR-Level Fuzzing for the LuaJIT Trace Optimizer

Aleksey Churkin[1], Sergey Bronnikov[2], and Sergey Kaplun[2]

[1] Moscow Institute of Physics and Technology (MIPT), Dolgoprudny, Russia
churkin.as@phystech.edu
[2] VK, Moscow, Russia

Abstract. Trace-based JIT compilers optimize aggressively and are validated almost exclusively by testing. We present ljopt, a bounded translation-validation tool for the LuaJIT tracing JIT compiler: it records the trace a program produces twice, unoptimized and optimized, translates both intermediate-representation (IR) traces into a single SMT-LIB formula asserting their inequivalence, and discharges it with an SMT solver—unsat proves the optimizer preserved the observable semantics of that trace, sat yields a counterexample. The encoding covers IEEE-754 arithmetic, LuaJIT's tagged values, versioned table memory, guards and side-exit snapshots, FFI 64-bit integers, and strings, and runs unmodified on both cvc5 and Z3. On top of the validator we built irfuzz, a fuzzer that synthesizes typed IR instruction streams and replays them through LuaJIT's real fold engine, using the validator itself as the bug oracle. Random and exhaustive enumeration modes reach 97.5 % line coverage of LuaJIT's optimizer (2449 of 2513 lines), with 16 of the missed lines argued unreachable individually, and the oracle's sensitivity is confirmed by fault injection. The tools re-detect six historical LuaJIT miscompilation bugs and found new ones, including a narrowing bug that makes `("abcdef"):sub(-0.5+3)` disagree with the interpreter; we authored the fix. A complementary end-to-end differential fuzzer, diffuzz, covers the compiler stages below the IR.

Keywords: translation validation · JIT compilation · SMT · compiler fuzzing · LuaJIT

1 Introduction

Tracing JIT compilers [6,3] compile only the hot paths a program actually takes, which makes their optimizers both unusually aggressive and unusually hard to test: the unit of compilation is not a function but a dynamically recorded instruction trace, specialized by guards that fall back to the interpreter. LuaJIT [14] is a mature, widely deployed example—it powers OpenResty, game scripting, and the Tarantool database—yet its optimizer has a steady history of miscompilation bugs, several of which survived in releases for years.

Verifying such a compiler wholesale, CompCert-style [9], is unrealistic: the implementation is hand-written C, evolves quickly, and its semantics is fixed

by the interpreter rather than by a specification. Translation validation [15,13] checks each individual compilation instead of the compiler: rather than proving the compiler correct, prove that *this* optimized program is equivalent to *this* unoptimized one. Alive2 [10] demonstrated that bounded translation validation with an SMT solver scales to LLVM. Tracing JITs are, in one respect, an even better fit: a trace is a straight-line program with explicit guards, so the control flow that makes whole-function validation hard largely disappears.

This paper presents ljopt, a translation validator for LuaJIT traces, and irfuzz, an IR-level fuzzer that turns the validator into a bug-finding oracle. Our contributions:

1. An SMT encoding of LuaJIT IR traces covering IEEE-754 doubles, tagged values as an algebraic datatype, versioned table memory with table-identity functions, guards and side-exit snapshots, bounded loop unrolling, FFI 64-bit integers, and strings (Sect. 3). The encoding is solver-portable and runs on both cvc5 [1] and Z3 [12].
2. irfuzz: a fuzzer that synthesizes typed IR instruction streams and replays them through LuaJIT's *actual* fold engine twice (optimizations off and on), then asks the validator whether the two results are equivalent (Sect. 4). Random and exhaustive modes reach 97.5 % line coverage of the optimizer, and fault injection confirms the oracle actually fires.
3. New bugs and a fix: a narrowing miscompile in `lj_opt_narrow.c` found by exhaustive enumeration (we wrote the patch), a loop-direction bug reported upstream as LuaJIT issue #1432, a segfault in `jit.util.tracek()`, and two pipeline bugs found by the companion differential fuzzer (Sect. 6).
4. An evaluation methodology for the validator itself: 35 regression cases built from historical LuaJIT bugs, checked to be sat on the buggy release and unsat after the upstream fix, plus a self-check mode that guards against vacuous proofs (Sect. 7).

ljopt is about 9,700 lines of Lua for the validator, 6,300 for the fuzzer, and small patches to LuaJIT itself; everything is open source.

2 Background: LuaJIT Traces

When a loop or function becomes hot, LuaJIT records the bytecode it executes into a linear SSA IR. Each IR instruction has an opcode, a result type (out of 23: doubles, 32- and 64-bit integers, strings, tables, ...), and two operands. Type checks and control decisions become *guards*: an instruction flagged as guarded aborts the trace at runtime if its condition fails, restoring the interpreter state described by the dominating *snapshot*—a map from Lua stack slots to IR references. The optimizer pipeline runs over this IR: the fold engine (algebraic rewrites, constant folding, CSE), alias analysis with load forwarding and dead-store elimination, loop-invariant code motion by peeling, allocation sinking, dead-code elimination, and *narrowing*, which rewrites double arithmetic into integer arithmetic where profitable. After optimization the trace is assembled to machine code.

A miscompilation at this level is observable as a wrong value in a slot, a wrong store to a table, or a guard that exits differently—exactly the three things our encoding compares.

3 Translation Validation for Traces

3.1 Recording twice

ljopt runs the input chunk twice in a sandboxed environment: once with `jit.opt.start(0)` and once with `jit.opt.start(3)` (all optimizations), both with `hotloop=1` so that even small chunks compile. A fork of LuaJIT's `jit/dump.lua` taps the compiler's introspection API (`jit.util.traceir`/`tracek`/`tracesnap`) to capture, for every recorded trace, the full IR with structured operands and all snapshots.[3] Traces from the two runs are matched by their starting location; the pair is then translated into one SMT-LIB formula [2].

3.2 Value and memory model

Lua values are dynamically typed, so cells are modeled as an algebraic datatype with six constructors:

```
(declare-datatypes ((MemCell 0))
  (((int-val (get-bv (_ BitVec 64)))
    (fp-val  (get-fp (_ FloatingPoint 11 53)))
    (str-val (get-str String))
    (tab-val (get-tab Int))
    (p32-val (get-p32-tab Int) (get-p32-idx MemCell))
    (nil-val))))
(define-sort MemPtr ()
  (Array Int (Array Int (Array MemCell MemCell))))
```

Numbers are IEEE-754 doubles in the SMT floating-point theory [4]; FFI integers are 64-bit bitvectors; strings use the SMT string theory; a table is an integer identity. A *reference into* a table (`p32-val`) carries both the table identity and the key cell, so string and numeric keys can never collide by accident. The formula is quantifier-free throughout.

Memory is a three-level array indexed $[version][table][key]$; every store bumps the version with a functional update, and loads read the current version. Three such arrays exist per trace pair: `shared_mem` is the unknown pre-trace heap, and `mem_unopt`, `mem_opt` are each constrained to equal it at version 0. Both traces therefore start from the *same* arbitrary heap, and the solver searches for any initial state that makes them diverge. Raw FFI memory is a separate byte-addressed array.

[3] One small upstream patch is required to expose snapshot positions; it was accepted into LuaJIT as part of pull request #1472.

Table identity by congruence. A subtle class of spurious results comes from naming tables. If each trace numbered the tables it loads independently, the same table reached through two paths would get two names, aliasing information would be lost, and either false alarms or—worse—vacuously true formulas follow. ljopt instead uses two uninterpreted functions: `tab_slot(Int)` maps a VM stack slot to the identity of the table found there (so `f(t,t)` correctly allows both arguments to alias), and `tab_uid(MemCell)` maps a cell to the table it would denote. Because both traces apply the same function to equal arguments, congruence makes them agree without ljopt having to know the actual pointer values. Tables allocated *inside* the trace get fresh symbolic identities constrained pairwise distinct and distinct from all pre-existing ones; escaped trace-local tables are paired across the two traces in allocation order and included in the final memory comparison.

3.3 Guards and snapshots

Every guarded instruction contributes a boolean "this guard held." Each snapshot accumulates its guards into a wide bitvector whose least-significant set bit identifies the first failing guard; the formula compares, across the two traces, (i) whether any guard fired at all, and (ii) the restored value of every slot of every snapshot, under a mapping that pairs snapshots across traces by position rather than by index (the optimizer renumbers them). Modeling guard failure as *data*—rather than as an assertion—is essential: guard elimination and guard strengthening are exactly the optimizations one wants to check.

Three disciplines for typed loads. When a trace loads a cell and asserts its type (e.g. `HLOAD` of a number), the encoding must pin the ADT constructor, otherwise the accessor returns an unconstrained value and the two traces diverge spuriously. But *how* it is pinned matters, and ljopt uses three different disciplines, chosen per case: a hard assertion where the recorder guarantees the type; a *guard condition* where the check is itself a trace exit, so a wrong type means "exit taken," not "formula inconsistent"; and, for table loads, a total decode $ite(is\text{-}tab(c), get\text{-}tab(c), tab_uid(c))$ whose fallback is congruent across traces. The danger being engineered around is vacuity: a single unsatisfiable assertion makes every trace pair "equivalent." An earlier version used a fresh counter as the fallback; the counters drifted apart between the two traces (their IR has different lengths) and produced false alarms—the congruent fallback fixed both problems at once.

3.4 Floating-point fidelity

The encoding is bit-precise for doubles, with deliberate deviations only where LuaJIT itself deviates from naïve IEEE semantics: `MIN`/`MAX` are encoded as ordered conditionals matching LuaJIT's `lj_vm_foldarith`, not as `fp.min`/`fp.max`, which SMT leaves underspecified on $(+0, -0)$; integer `MOD` is floor-mod; shift counts are

masked as on x86; numeric equality uses `fp.eq`, while the final state comparison uses structural equality *relaxed on NaN sign*, because LuaJIT folds $-1.0 \cdot x$ into a sign-bit XOR, which flips the sign of a NaN where `mulsd` would preserve it. $+0.0$ and -0.0 remain distinguished: Lua code can observe the difference through $1/x$. Transcendentals, `POW`, string–number coercions, table rehash sizes, and external calls are uninterpreted functions—equivalence is then checked up to congruence, which is the right contract for optimizations that may reorder but not re-evaluate such calls.

3.5 Loops, and what "bounded" means

LuaJIT's loop optimization peels one iteration and inserts PHIs. Rather than encode PHIs, ljopt unrolls: the unoptimized trace body n times against the optimized body's peel plus $n-1$ iterations (default $n=4$), duplicating snapshots per iteration. Like Alive2, the result is *bounded* translation validation: a divergence that only manifests after more iterations is missed.

Instructions the encoding does not model are handled by *transitive weakening*: the unknown node and everything data-dependent on it are replaced by unconstrained values, and their guards default to "held." This can only weaken the check—it can never invent a divergence—and a strict mode turns any weakening into a hard error. Out of LuaJIT's 99 IR opcodes, 17 cannot appear as instructions by construction (constants, markers), and ljopt models roughly 76 of the remaining 82 via 202 typed handlers.

3.6 Guarding against vacuous proofs

An equivalence checker that emits an inconsistent formula reports unsat—"everything verified"—and no test based on correct compilations will ever notice. Three fixed bugs of exactly this shape motivated a standing defense: a self-check mode drops the inequivalence disjunct and requires the remaining constraints to be *satisfiable* (the recorded execution must be a model of its own trace). The historical bug corpus (Sect. 7) provides the complementary direction: formulas that must be sat.

4 irfuzz: Fuzzing the Optimizer at the IR Level

Translation validation checks the traces your test programs happen to produce. To search the optimizer's input space directly, irfuzz inverts the setup: it *synthesizes* IR and uses the validator as the oracle.

4.1 Replaying synthetic IR through the real optimizer

A 335-line patch adds `jit.util.irfuzz` to LuaJIT: it takes a typed instruction stream (opcodes, result types, operand kinds and payloads) and replays it twice through the *production* fold engine—once with all optimizations disabled and

once at `-O3`, optionally followed by the loop, DCE, and sinking passes—returning both resulting IR buffers. No bytecode, recorder, or assembler is involved: the unit under test is precisely the optimizer. A decoder rebuilds each buffer into the exact structure the trace recorder would have produced, so the validator cannot tell synthetic IR from recorded IR, and the comparison entry point is *the same function* used for recorded traces. `unsat` means the optimizer preserved observable state on this input; `sat` is a candidate miscompile or a modeling gap, triaged by replaying the seed.

4.2 Generation

Random mode builds streams of `SLOAD` inputs, constants, and a body of arithmetic, bitwise, comparison, and table operations, using a local deterministic PRNG so seeds reproduce across builds. Operand selection is *blacklist*-based: only combinations the recorder provably never emits, or that hit a known modeling gap, are excluded—a whitelist would only ever exercise what the model already covers and could never surface a new gap. Constants are exactly the literals LuaJIT's fold rules special-case (0.0, -0.0, 1.0, 2.0, 0.5, powers of two, int patterns like $2^{31}-1$), plus deliberately non-special controls, so "no fold applies" paths are exercised too.

Exhaustive mode enumerates, seedlessly and resumably, every operator chain of a given depth over a fixed leaf set—$|ops|^d \cdot |leaves|^{d+1}$ trace shapes, both operand orders for non-commutative operators. Depth 2 is exhaustively tractable; depth 3 ($\approx$12.5M shapes for doubles) is sampled with a limit. Seventeen targeted enumerators cover specific pass surfaces: mixed-type conversion chains (the 47 `CONV` fold rules), comparison guards (observable only through the exit bitvector), aliasing shapes for load forwarding and dead-store elimination, the narrowing pass, allocation sinking, string buffers, bounds-check elimination, upvalues, and FFI pointer arithmetic.

Two pre-solver filters are provably sound: optimized output byte-identical to the input is skipped, as are pairs differing only by operand swaps of provably commutative operators (the fold engine canonicalizes $k + x$ to $x + k$; a solver otherwise re-proves FP commutativity from scratch on every such seed). `MIN`/`MAX` are deliberately excluded from the swap filter—they are *not* commutative on $(+0, -0)$.

Loop mode feeds generator output i back into the slot that `SLOAD` i reads, making every value loop-carried, and runs LuaJIT's loop pass on the optimized side while the validator unrolls both. A type-respecting permutation assigns each slot a value of its own type; without it, the two traces model the resulting type change differently and report divergences the optimizer never caused (the debugging trail ends at the memorable expression $-2^{31} \cdot -0.0$).

Table 1. Line coverage of LuaJIT's optimizer under the irfuzz sweep.

file	lines	hit	%
`lj_opt_fold.c`	1241	1209	97.4
`lj_opt_mem.c`	572	554	96.9
`lj_opt_loop.c`	245	239	97.6
`lj_opt_dce.c`	35	35	100.0
`lj_opt_sink.c`	148	145	98.0
`lj_opt_narrow.c`	272	267	98.2
total	2513	2449	97.5

4.3 Solvers

Queries run against standalone solver binaries—one crash or timeout cannot take down a campaign—with verdicts matched line-exactly. cvc5 is the default (`-arrays-exp`, per-query time limits). On 180 byte-identical queries drawn evenly from six generation modes, at a 30 s limit, cvc5 closed 166 and Z3 134; the union is 171, with 37 closed only by cvc5 and 5 only by Z3. The two fail differently rather than one dominating: Z3's median is three times faster (0.33 s against 0.94 s) yet it abandons more than a quarter of the queries, so its mean is nearly twice cvc5's—a median alone would rank the solvers backwards. Extending the comparison to 1470 queries over seven modes, the two *never* returned contradictory verdicts, and neither produced a model its own checker rejected; they differ only in strength, and 225 of the 276 queries cvc5 closes alone come from the narrowing mode, whose `fp.to_sbv` sinks are the hardest construct in the encoding for both solvers. Making the encoding portable required removing Z3-specific constructs (`fp.to_ieee_bv`, rotation builtins, constant arrays)—portability doubled as a soundness review.

4.4 Coverage, and validating the fuzzer itself

A full sweep of all generation modes (8000 traces per mode, with and without loop replay) against a gcov-instrumented LuaJIT covers **2449 of 2513 lines, 97.5 %**, of the optimizer (Table 1).

Of the 64 missed lines, 16 are argued unreachable individually (assertion-guarded `default` arms, fold-table entries no rule can reach, arms dead when folding is disabled); each remaining line has a documented reason.

Coverage alone does not show the oracle can *fire*, so irfuzz is validated by mutation: injecting a fault into a fold rule must flip a clean sweep to sat. Injecting an off-by-one into the string-length fold turns a clean sweep (0 sat) into all-sat; forcing the alias analysis to treat may-alias as must-alias flips 18 of 120 aliasing shapes. The negative results are reported with equal weight: stubbing out the sinking heuristic changes nothing the decoder can see—sinking is *covered but not verified*, since its effect lives in a field the introspection API does not expose—and

one generic sweep failed to transport four injected faults at all, so it is classified as coverage-only, not as an oracle.

A third, syntactic oracle. One historical bug class (LuaJIT #524) corrupts the *type annotation* of an emitted `CONV` while its semantics—keyed on the conversion mode—stays right; the damage happens in the backend, below the SMT abstraction, and no semantic oracle at the IR level can see it. A one-page lint ("a `CONV`'s result type must equal its mode's destination type") runs on every fuzzed trace and is reported independently. Layering oracles with different blind spots is cheap and, in our experience, necessary.

5 diffuzz: Differential Testing Below the IR

irfuzz reasons about IR semantics, so it is blind to everything the compiler does after the IR: register allocation, snapshot restoration, trace linking. diffuzz closes that gap from the other side: it generates random Lua programs, runs them under the interpreter and under 16 JIT configurations (optimization levels, hot-counter settings, and each pass individually disabled), and compares outputs. A configuration with one pass disabled must never change results, so a divergence under `-O3,-dce` is as much a defect as one at `-O3`—and the per-pass rows localize the failure without a separate bisection.

The design constraint is the oracle: every generated value is an exact integer reduced modulo 1000003. Floating point ruins end-to-end differential testing—a one-ULP interpreter/JIT difference is usually a legitimate `libm` difference—and an earlier float-based generator produced 679 "mismatches" over 15k seeds, every one undefined behavior rather than a bug. The exact-integer generator is also the more sensitive one: against a deliberately injected alias-analysis fault it flags 15 of the first 60 seeds where the float version caught 2.

6 Findings

A new narrowing miscompile (fix authored). The exhaustive narrowing enumeration found that `narrow_conv_backprop` sinks an *unguarded* number-to-integer conversion inside an addition: $int(x + double(i))$ becomes $int(x) + i$, so rounding happens before the addition instead of after. The bug is reachable from plain Lua through every `IRCONV_ANY` consumer:

```
("abcdef"):sub(-0.5 + 3)  --> "cdef"  (interpreter: "bcdef")
bit.tobit(0.5 + 1)        --> 1       (interpreter: 2)
```

We wrote the fix—allow the sunk conversion only for guarded conversion modes—and validate it in CI: the reproducers fail on eight unpatched LuaJIT builds and pass on the patched one.

Other new findings. (i) A wrong loop direction for a `-0` step, reported upstream as LuaJIT issue #1432. (ii) A segmentation fault in `jit.util.tracek()`: the payload half of a 64-bit constant is misread as an instruction, and one byte value turns it into an out-of-bounds dereference; reachable from plain Lua, confirmed on upstream HEAD. (iii) By diffuzz: a store lost at default `-O3`—an `HSTORE` inside a nested loop is dropped by the interaction of five passes (forwarding, DSE, CSE, fold, loop), so a later read observes a stale value. (iv) By diffuzz: with DCE disabled, a loop-invariant value referenced only by side-exit snapshots is assigned a register the loop body reuses, so the exit restores an unrelated variable—a register-allocator bug on which ljopt correctly answers unsat, since the IR is equivalent; the defect is introduced below the IR. This pair of findings is the concrete argument for layering irfuzz and diffuzz.

Bugs found in the oracle itself. Fuzzing also debugged ljopt: encoding `MIN`/`MAX` as `fp.min`/`fp.max` flagged a correct fold of $\min(-0.0, +0.0)$; unmasked shift counts and structural FP equality caused similar false alarms. We report these because they are the expected failure mode of any verification tool used as a fuzzing oracle, and the fault-injection discipline of Sect. 4.4 is what kept them from silently becoming false *negatives*.

7 Evaluation

Historical bug corpus. The strongest evidence that the validator's sat verdicts mean something is behavioral: 35 regression cases reconstruct known LuaJIT miscompilation bugs (among them issues #505, #524, #783, #784, #791, #794, #797, #980, #994, #1069, #1079, #1133, #1194, #1236, #1262). Each case is run against two pinned LuaJIT builds—one just before the upstream fix, one just after—and the harness requires sat on the buggy build and unsat on the fixed one. Six of these (#794, #797, #817, #1069, #1194, #1236) are detected by the SMT oracle itself, i.e. the encoding sees the semantic divergence; the others reproduce behaviorally and pin the encoding's handling of the affected constructs.

Verification throughput. The end-to-end suite translates 90 Lua chunks (arithmetic, tables, globals, strings, upvalues, FFI, bounds checks) and requires each resulting formula to parse and prove unsat; 93 per-instruction cases check single opcodes against concretely known results. On a curated 149-file corpus of generated Lua programs, a sweep produced 625 unsat verdicts, 9 sat (all triaged to one known modeling gap, below) and 25 timeouts. Fuzzing sweeps at the IR level ran 5,000 random arithmetic seeds and 1,600 table seeds with zero unexplained sat verdicts on clean LuaJIT builds.

Limitations. ljopt is bounded (loop unrolling, default depth 4) and IR-level: the assembler and register allocator are out of scope by construction, which finding (iv) above makes concrete. Memory is currently compared at its final version

only, so a dead store observable at an intervening side exit can be missed—we constructed this false negative deliberately (by weakening DSE's guard check and watching the validator stay `unsat`) and the fix direction is to compare memory per snapshot at its version. Known false-positive sources are equally documented: repeated or dead stores to an escaping table can produce a spurious `sat` from the memory-diff encoding, and uninterpreted abstractions check equivalence only up to congruence (two `math.random` calls compare equal). Solver-side, the formulas need cvc5 ≥ 1.2 (older versions answer some of them *incorrectly*), and Z3's Horn backend is unusable here for lack of FP and string sorts.

8 Related Work

Translation validation originates with Pnueli et al. [15] and Necula [13]; Alive and Alive2 [11,10] made bounded SMT-based validation practical for LLVM and are the closest relatives of this work. ljopt differs in its target's shape: traces are straight-line and guard-structured, memory is a dynamically typed heap of tables rather than typed LLVM memory, and the semantics baseline is an interpreter, not a language reference. Compiler fuzzing by differential execution goes back to Csmith [17] and EMI [8]; JIT-directed fuzzers such as Fuzzilli and FuzzJIT [7,16] generate source programs and rely on crashes or differential behavior. irfuzz's distinguishing choice is to generate *IR* and use a translation validator as the oracle, which detects silent wrong-value optimizations that neither crash nor happen to be exercised observably by a source-level test—while measuring, by line coverage and fault injection, how much of the optimizer the campaign actually judges. Mutation-based validation of the toolchain itself follows the spirit of solver fuzzing [5].

9 Conclusion

ljopt shows that bounded translation validation is a practical fit for a production tracing JIT: the trace shape removes most control-flow complexity, and a careful ADT-plus-versioned-arrays heap encoding handles a dynamically typed language. Used as a fuzzing oracle over synthesized IR, it judges 97.5 % of LuaJIT's optimizer lines and has found and fixed real miscompiles. The tool, the fuzzers, the bug corpus, and all coverage and fault-injection protocols are open source.

References

1. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 13243, pp. 415–442. Springer (2022)

2. Barrett, C., Fontaine, P., Tinelli, C.: The SMT-LIB standard: Version 2.6. Tech. rep., Department of Computer Science, The University of Iowa (2017), available at `https://smt-lib.org`
3. Bolz, C.F., Cuni, A., Fijałkowski, M., Rigo, A.: Tracing the meta-level: PyPy's tracing JIT compiler. In: Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS). pp. 18–25. ACM (2009)
4. Brain, M., Tinelli, C., Rümmer, P., Wahl, T.: An automatable formal semantics for IEEE-754 floating-point arithmetic. In: IEEE Symposium on Computer Arithmetic (ARITH). pp. 160–167 (2015)
5. Brummayer, R., Biere, A.: Fuzzing and delta-debugging SMT solvers. In: International Workshop on Satisfiability Modulo Theories (SMT). pp. 1–5. ACM (2009)
6. Gal, A., Eich, B., Shaver, M., Anderson, D., Mandelin, D., Haghighat, M.R., Kaplan, B., Hoare, G., Zbarsky, B., Orendorff, J., Ruderman, J., Smith, E.W., Reitmaier, R., Bebenita, M., Chang, M., Franz, M.: Trace-based just-in-time type specialization for dynamic languages. In: Programming Language Design and Implementation (PLDI). pp. 465–478. ACM (2009)
7. Groß, S., Koch, S., Bernhard, L., Holz, T., Johns, M.: FUZZILLI: Fuzzing for JavaScript JIT compiler vulnerabilities. In: Network and Distributed System Security Symposium (NDSS) (2023)
8. Le, V., Afshari, M., Su, Z.: Compiler validation via equivalence modulo inputs. In: Programming Language Design and Implementation (PLDI). pp. 216–226. ACM (2014)
9. Leroy, X.: Formal verification of a realistic compiler. Communications of the ACM **52**(7), 107–115 (2009)
10. Lopes, N.P., Lee, J., Hur, C.K., Liu, Z., Regehr, J.: Alive2: Bounded translation validation for LLVM. In: Programming Language Design and Implementation (PLDI). pp. 65–79. ACM (2021)
11. Lopes, N.P., Menendez, D., Nagarakatte, S., Regehr, J.: Provably correct peephole optimizations with Alive. In: Programming Language Design and Implementation (PLDI). pp. 22–32. ACM (2015)
12. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 4963, pp. 337–340. Springer (2008)
13. Necula, G.C.: Translation validation for an optimizing compiler. In: Programming Language Design and Implementation (PLDI). pp. 83–94. ACM (2000)
14. Pall, M.: The LuaJIT project. `https://luajit.org`, accessed August 2026
15. Pnueli, A., Siegel, M., Singerman, E.: Translation validation. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 1384, pp. 151–166. Springer (1998)
16. Wang, J., Zhang, Z., Liu, S., Du, X., Chen, J.: FuzzJIT: Oracle-enhanced fuzzing for JavaScript engine JIT compiler. In: USENIX Security Symposium. pp. 1865–1882 (2023)
17. Yang, X., Chen, Y., Eide, E., Regehr, J.: Finding and understanding bugs in C compilers. In: Programming Language Design and Implementation (PLDI). pp. 283–294. ACM (2011)